Take home question

Product Context & User Needs

ExampleHR has a module that serves as the primary interface for employees to request time off. However, the **Human Capital Management (HCM)** system (like Workday or SAP) remains the "Source of Truth" for employment data.

The Problem: Keeping balances synced between two systems is notoriously difficult. If an employee has 10 days of leave and requests 2 days on ExampleHR, we need to ensure the HCM agrees they have the balance, and we must handle cases where the HCM balance changes independently (e.g., a "work anniversary" bonus).

User Persona:

- **The Employee:** Wants to see an accurate balance and get instant feedback on requests.
- **The Manager:** Needs to approve requests knowing the data is valid.

The task

You are tasked with building the **Time-Off Microservice**. Your goal is to manage the lifecycle of a time-off request and maintain balance integrity.

Here are a few interesting challenges

- ExampleHR is not the only system that updates HCM; for example on work anniversary or start of the year, our customers' employees may get a refresh of time off balances
- HCM provides a realtime API for getting or sending time off values (e.g. 1 day for locationId X for employeeId Y)
- HCM provides a batch end point that would send the whole corpus of time off balances (with necessary dimensions) to ExampleHR
- We can count on HCM to send back errors to us in case we want to file a time-off against an invalid combination of dimensions or against insufficient balance HOWEVER this may not be always guaranteed; we want to be defensive about it.

We want to design a backend system, micro-service, that would create all the necessary REST (or GraphQL) endpoints for handling time off balances and syncing them up with HCMs.

What your work will be measured against

- **Eng Spec**: A well written Technical Requirement Document (TRD), with challenges listed, a suggested solution, and good analysis of alternatives considered.

- **Test Suite:** Since you are using **Agentic Development**, the value of your work lies in the rigor of your tests. Make your choice on the type of tests, the goal is to make sure the system is robust and can guard against regressions from future development.
- **Deliverables:**
    - Your TRD
    - Your code in a repository on github
    - Your test cases and proof of coverage

Guides

- Go all in with agentic development; do not write even a single line of code, but be very picky and precise about your TRD and be very thorough with your test cases.
- Create mock endpoints (you may want to deploy real mock servers for them with some basic logic to simulate balance changes) for the HCM as part of your test suite
- Develop with NestJs and SQLite.
- Assume balances are per-employee per-location